SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Chennuri sumedha
Register Number	1971260029
Course Code & Name	C- programming (CSA0210)
Slot	Slot - c
Assignment / Task Title	Design, implement and validate a reliable billing program for a rural electricity co - operative
GitHub Repository URL	https://github.com/sumedha-2805/C-Programming.git
Submission Date	31-10-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK1

Task / Problem Statement

UNDERSTAND THE PROBLEM AND IDENTIFY THE DATA

Aim

Theaim of this task is to understand the electricity billing problem and identify the input data,output data, constants, and validation rules required to develop a reliable C program.

Concept / Theory

The Grama JyothiRural Electricity Co-operative needs a C program to calculate the electricity bill for each household. The program uses the previous meter reading and the current meter reading to determine the electricity units consumed.

The program must also handle meter rollover, apply the correct tariff rates, include the fixed charge, and display the final bill accurately.

Procedure / Algorithm

1. Read the problem statement carefully.
2. Identify the data required for calculating the electricity bill.
3. Identify the input values, such as household ID and meter readings.
4. Identify the expected outputs, including units consumed and total bill.
5. Define the fixed charge and electricity tariff rates.
6. Identify validation rules for meter readings.
7. Plan how the program will handle meter rollover and calculate the bill.

Program / Source Code

```c
#include <stdio.h>
#define METER_MAX 65535U
#define METER_RANGE 65536U
int main(void)
{unsigned int previous, current, units;
 printf("Enter previous meter reading: ");
 if (scanf("%u", &previous) != 1 ||
 previous > METER_MAX) {
 printf("Invalid previous reading.\n");
        return 1;
    }
printf("Enter current meter reading: ");
    if (scanf("%u", &current) != 1 ||
        current > METER_MAX) {
        printf("Invalid current reading.\n");
        return 1;
    }
   if (current >= previous)
        units = current - previous;
    else
        units = METER_RANGE - previous + current;

    printf("Units consumed: %u\n", units);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

The programcalculateselectricity consumption by comparing the previous meter reading with thecurrentmeterreading.Before performing the calculation, it checks whether both readingsarewithinthevalidrange of 0 to 65,535. When the current reading is greater than orequaltothepreviousreading, the program calculates consumption by subtracting the previousreadingfromthecurrent reading. When the current reading is smaller, the programassumesthatthemeter has completed one cycle and rolled over to zero. It then calculatestheconsumptionusing the meter range of 65,536 . For example, when the previousreadingis1200andthe current reading is 1530, the consumption is 330 units. In the rollovercase,whentheprevious reading is 65400 and the current reading is 120, the consumptionis256units.Theprogram uses constants for the maximum meter reading and meterrange,whichmakesthe code easier to understand and maintain. It also validatesuserinputtopreventinvalid meter readings from being processed.

Result

The Cprogram was implemented to calculate electricity units consumed from previous and current meter readings. It successfully calculates consumption during normal meter operation and handles meter rollover correctly. The program also rejects readings outside the permitted range. Therefore, the program meets its objective of calculating electricity consumption accurately for the tested cases.

TASK 2

Task / Problem Statement

CHOOSE DATA TYPES AND HANDLE METER ROLLOVER

Aim

To select suitable data types for the electricity billing program and calculate electricity consumption correctly, including meter rollover.

Concept / Theory

Explain the Pythonconcepts used in this task in your own words.

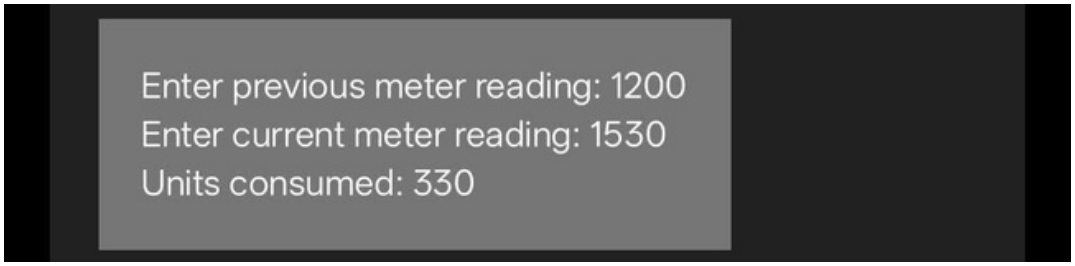
Procedure / Algorithm

1. Identify the variables required for the electricity billing program.
2. Select suitable C data types based on the range and purpose of each variable.
3. Store the previous and current meter readings as unsigned integers.
4. Validate that both readings are between 0 and 65,535.
5. Compare the current reading with the previous reading.
6. If the current reading is greater than or equal to the previous reading, subtract the previous reading from the current reading.
7. If the current reading is smaller, calculate consumption by accounting for meter rollover.
8. Display the totalunits consumed.

Program / Source Code

```
#include <stdio.h>
#define METER_MAX 65535U
#define METER_RANGE 65536U
int main(void)
{unsigned int previous, current, units;
  printf("Enter previous meter reading: ");
    if (scanf("%u", &previous) != 1 || previous > METER_MAX) {
        printf("Invalid previous reading.\n");
        return 1;
    } printf ("Enter current meter reading: ");
    if (scanf ("%u", &current) != 1 ||
    current > METER_MAX) {
    printf("Invalid current reading.\n");
    return 1;
    } if (current >= previous) {
        units = current - previous;
    } else {
  units = METER_RANGE - previous + current;
    }printf("Units consumed: %u\n", units);
return 0;
}
```

Output Screenshot



Analysis / Discussion

Choosing suitable datatypes helps the program store meter readings and calculate consumption correctly.Since meter readings are non-negative, an unsigned integer is appropriate for storingthem.During normal operation, consumption is calculated by subtracting the previousreading from the current reading. When the meter reaches its maximum reading androlls over to zero, the program uses the full meter range of 65,536 to calculate the unitsconsumed.For example, the normal readings of 1200 and 1530 produce 330 units. Intherollover example, a previous reading of 65400 and a current reading of 120 produce256 units. Input validation prevents readings outside the permitted range from being accepted.

Result

The Cprogram successfully demonstrates suitable data type selection and electricity consumption calculation. It correctly calculates units for both normal meter readings and meterrollover and rejects invalid meter readings.

TASK 3

Task / Problem Statement

DEFINE CONSTANTS AND CALCULATE ELECTRICITY BILLING

Aim

To define suitable constants and develop a C program to calculate the electricity bill using tariffslabs and a fixed charge.

Concept / Theory

Constants are fixedvalues that do not change during program execution.InC,the "#define" preprocessor directive can be used to define symbolic constants.Using constants makesaprogram easier to understand, maintain, and modify.Inthiselectricity billing system, thetariff rates and fixed charge are defined as constants. Thefirst 100 units

are free, units from 101 to 200 are charged at ₹2.25 per unit, and units above 200 are charged at ₹4.50 per unit. A fixed charge of ₹50 is added to every bill.The electricity bill is calculated by applying the appropriate rate to each tariff slab. To avoid rounding errors when calculating money, the program stores amounts in paise, where 100 paise equals ₹1.

Procedure / Algorithm

1. Start the program.
2. Define constants for the free units, tariff limit, tariff rates, and fixed charge.
3. Read the number of electricity units consumed.
4. Validate the input.
5. Initialize the bill with the fixed charge.
6. Calculate the charge for units above 200, if applicable.
7. Calculate the charge for units between 101 and 200, if applicable.
8. Add the slab charges to the fixed charge.
9. Display the total electricity bill in rupees and paise.
10. Stop the program

Program / SourceCode

```
#include <stdio.h>
#define FREE_UNITS 100
#define SLAB2_LIMIT 200
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define  FIXED_CHARGE_PAISE  5000L
int main(void)
{unsigned int units;
    long bill_paise = FIXED_CHARGE_PAISE;
  printf("Enter units consumed: ");
 if (scanf("%u", &units) != 1) {
      printf("Invalid input.\n");
      return 1;
   }if (units > SLAB2_LIMIT) {
      bill_paise +=
          (long)(units - SLAB2_LIMIT) * RATE3_PAISE;
   }if (units > FREE_UNITS) {
      unsigned int slab2_units =
          (units < SLAB2_LIMIT)
          ? units - FREE_UNITS
          : SLAB2_LIMIT - FREE_UNITS;
```

```
 bill_paise +=
          (long)slab2_units * RATE2_PAISE;
    }printf("Units consumed: %u\n", units);
    printf("Fixed charge: Rs. 50.00\n");
    printf("Total electricity bill: Rs. %ld.%02ld\n",
  bill_paise / 100, bill_paise % 100);
return 0;
}
```

Output Screenshot



Analysis / Discussion

The program uses symbolic constants to represent tariff limits, rates, and the fixed charge. This avoids repeating unexplained numeric values throughout the code.

The first 100 units do not incur an energy charge. For consumption between 101 and 200 units, the program charges ₹2.25 for each unit in that slab. For consumption above 200 units, it charges ₹4.50 for each additional unit. The fixed charge is included regardless of consumption.For 150 units, the energy charge is (50 \times ₹2.25 = ₹112.50). Adding the fixed charge of ₹50 gives a total bill of ₹162.50.For 350 units, the energy charge is (100 \times ₹2.25 + 150 \times ₹4.50 = ₹900). Adding the fixed charge gives ₹950.00. Calculating in paise helps avoid floating-point rounding errors in these monetary calculations.

Result

The C program successfully calculates electricity bills using the specified tariff slabs and fixed charge. The tested results for 150 units and 350 units are ₹162.50 and ₹950.00, respectively.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each testpassed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1 TCO1	0 units	Total bills: ₹50.00	₹50.00	pass
2 TCO2	100 units	Total bills: ₹50.00	₹50.00	pass
3 TCO3	150 units	Total bills: ₹162.50	₹162.50	pass
4 TCO4	200units	Total bills: ₹275.00	₹275.00	pass
5 TCO5	350units	Total bills: ₹950.00	₹950.00	pass

Final Submission Checklist

- □ Student and course details completed
- □ Task statement, aim, theory, and algorithm included
- □ Program code is readable and runs successfully
- □ Output screenshot included
- □ Analysis and result written in own words
- □ Test cases completed where applicable
- □ GitHub repository link added if required

TASK 4 — USE MACROS SAFELY

Task / ProblemStatement

Writemacrosforsmall repeated calculations, such as converting rupees to paise and finding the larger of two values. Explain preprocessor expansion, common macro pitfalls, and when an ordinary function is safer.

Aim

Todefine safe function-like macros, inspect their expansion using **gcc -E**, and understand the risks of missing parentheses and repeated argument evaluation.

Concept / Theory

Amacrois handledby the C preprocessor before compilation. A function-like macro performs textual substitution. Every parameter occurrence is replaced by the corresponding argument, so arguments with side effects can be evaluated more than once. Parentheses around each parameter and the complete replacement expression prevent precedence errors. Unlike a function, a macro has no type checking and may duplicate code at every use.

Procedure / Algorithm

- Definemacrosfor rupees-to-paise conversion, square and maximum.
- Put parentheses around every macro parameter and the whole replacement expression.
- Use ordinary variables as arguments when demonstrating the safe version.
- Compile and run the program to verify the calculations.
- Run gcc -E on the source file and inspect the expanded expressions.

Program / Source Code

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((r) * 100)
#define   SQUARE(x)         ((x) * (x))
#define MAX(a, b)           (((a) > (b)) ? (a) : (b))

int main(void)
{
    int rupees = 12;
    int a = 4, b = 9;

    printf("Paise = %d\n", RUPEES_TO_PAISE(rupees));
    printf("Square = %d\n", SQUARE(a + 1));
    printf("Maximum = %d\n", MAX(a, b));
    return 0;
}
```

Output

```
Paise = 1200
Square  =  25
Maximum = 9
```

Macro Pitfalls and gcc -E Evidence

1.Missing parentheses: IfSQUAREis written as#define SQUARE(x) x*x, then SQUARE(a+1) expands to a+1*a+1. For a=4, this becomes 4+4+1 = 9, not 25. The safe definition expands to ((4+1)*(4+1)).

2. Repeated evaluation: MAX(i++, j) may evaluate i++ once in the comparison and again in the selected expression. This can change i more than once and make the result surprising. Avoid side-effect expressions as macro arguments.
3. gcc -E: Save the program as task4.c and run gcc -E task4.c. The preprocessed output shows the macro calls replaced by expressions such as (((a + 1) * (a + 1))). Header contents and line markers may also appear.

When a Function Is Better

Usea function when arguments may have side effects, type checking is important, the operation is large, or debugging is easier with a normal function. A static inline function is often a safer alternative: it has function semantics and the compiler may inline it.

Alternative Concept and Complexity Analysis

Replacing SQUARE and MAXwithstaticinline functions gives one function definition and a call at each use. A normal call may have a small call/return overhead; an optimizing compiler may inline it and remove that overhead. A macro is expanded at every use, so generated code can grow roughly with the number of uses k and the size of the replacement expression. An inline function keeps one source definition, although the compiler controls whether machine code is duplicated.

Result

Safe macros were defined using complete parentheses. Preprocessor expansion and the risks of operator precedence and repeated argument evaluation were explained. Functions are preferable when type safety and single evaluation are required.

TASK 5 — ORGANISE THE PROGRAM WITH HEADER FILES AND CONDITIONAL COMPILATION

Task / ProblemStatement

Organise constantsandfunctionprototypes in a header file, protect the header with include guards, and add a DEBUG mode that prints intermediate values only when compiled with -DDEBUG.

Aim

Toseparate declarations from implementation, prevent repeated header inclusion, and control debugging statements at compile time.

Concept / Theory

AC program canbe organised into header files (.h), source files (.c), global declarations, main(), and other functions. An include guard uses #ifndef, #define and #endif so the same header is not processed repeatedly in one translation unit. #ifdef DEBUG includes debugging code only when DEBUG is defined. Angle brackets normally search system include directories, while quotation marks search the current source directory first.

Procedure / Algorithm

• Create a header file containingconstants and function prototypes.

• Add an include guard around the header contents.

• Write function definitions in a separate source file.

• Use #ifdef DEBUG around intermediate printf statements.

• Compile normally and again with gcc -DDEBUG to compare the outputs.

Header File — bill.h

```
#ifndef BILL_H
#define BILL_H

#define TAX_RATE 0.05

double calculate_total(double amount);

#endif
```

Source File — bill.c

```
#include <stdio.h>
#include "bill.h"

double calculate_total(double amount)
{
    double tax = amount * TAX_RATE;
#ifdef DEBUG
    printf("[DEBUG] Amount = %.2f\n", amount);
    printf("[DEBUG] Tax = %.2f\n", tax);
#endif
    return amount + tax;
}
```

Main File — main.c

```
#include <stdio.h>
#include "bill.h"

int main(void)
{
    double amount = 200.0;
    printf("Total = %.2f\n", calculate_total(amount));
    return 0;
}
```

Compilation and Output

```
Normal build:
gcc main.c bill.c -o bill
./bill
Total = 210.00

Debug build:
gcc -DDEBUG main.c bill.c -o bill_debug
./bill_debug
[DEBUG] Amount = 200.00
[DEBUG] Tax = 10.00
Total = 210.00
```

The include guard prevents repeated declarations if bill.h is included more than once. The DEBUG messages appear only in the debug build.

Alternative Concept and Complexity Analysis

Ifdebugging were controlledby a run-time variable, eachdebug statement would require a condition check whenever execution reaches it. The check is usually small, but remains in the executable and is performed during normal runs. With #ifdef DEBUG, the preprocessor removes the debug statements completely in a normal build, so there is no run-time test and no debug-print code in that executable. The debug build includes the statements.

Result

Theprogram was divided into a header and source files. Include guards prevent repeated inclusion, and conditional compilation enables debugging output only when requested.

TASK 6 — READ AND PRINT DATA CORRECTLY

Task / ProblemStatement

Design inputand outputusing scanf() and printf(). Use correct format specifiers, validate input using scanf()'s return value, and display an integer amount of paise as rupees and paise.

Aim

Toreadvalues safely, match format specifiers to data types, detect invalid input, and print a correctly formatted currency amount.

Concept / Theory

The format specifier must match the variable type. Common examples are %d for int, %u for unsigned int, %ld for long, %lld for long long, %f for float output, %lf for double input with scanf, %c for char and %s for a character array. For printf, %f prints a double value. scanf() returns the number of input items successfully assigned; checking this value helps detect invalid input.

Procedure / Algorithm

- Declare aninteger variabletostore the amount in paise.
- Read the value with scanf("%lld", &paise;).
- Check whether scanf returns 1; otherwise display an invalid-input message.
- Separate rupees using integer division by 100 and paise using the remainder operator.
- Print the result with two digits after the decimal point.

Program / Source Code

```
#include <stdio.h>

int main(void)
{
    long long paise;
    long long rupees, remaining_paise;

    printf("Enter amount in paise: ");
    if (scanf("%lld", &paise) != 1 || paise < 0) {
        printf("Invalid input\n");
        return 1;
    }

    rupees = paise / 100;
    remaining_paise = paise % 100;

    printf("Amount = Rs. %lld.%02lld\n", rupees, remaining_paise);
    return 0;
}
```

Output

```
Enter amount in paise: 16250
Amount = Rs. 162.50
```

Format Specifiers and Invalid Input

Type / purpose	Specifier
int	%d
unsigned int	%u
long	%ld
long long	%lld
float input / printf floating value	%f
double input with scanf	%lf
char	%c
string (char array)	%s

Using %d for a long value is a format mismatch and causes undefined behaviour. Always use the matching specifier. For a string, limit input width (for example, %29s for char name[30]) to avoid writing past the array.

Alternative Concept and Complexity Analysis

Analternativeis to read a wholeline with fgets() and thenconvert it using sscanf() or strtol(). Processing a line of length L takes O(L) time and requires O(L) buffer space. Direct scanf() is concise but can leave invalid characters in the input stream, making recovery awkward. fgets() captures the whole line, allowing the program to reject or clear the entire invalid entry before asking again. strtol() also allows checking where conversion stopped and whether the value overflowed.

Result

Theprogram checks scanf()'s return value, rejects invalid or negative input, and formats 16250 paise as Rs. 162.50.

TASK 7 — CONTROL TYPE CONVERSION AND PRECISION

Task / ProblemStatement

Identifywherevalues of different types are combined. Explain implicit conversion, integer division, explicit casting, narrowing a long value to int, and rounding when converting fractional charges to paise.

Aim

Tounderstand how C converts values during expressions and how to avoid precision loss in currency calculations.

Concept / Theory

Inmixed arithmetic,C applies the usual arithmetic conversions. Smaller integer types are promoted, and when an integer and floating-point value are combined, the integer is generally converted to floating point. Integer division discards the fractional part. An explicit cast requests a conversion, but converting a large long value to int may lose information if the value is outside the int range. For currency, storing integer paise avoids many floating-point representation errors.

Procedure / Algorithm

• Identifyeachexpression that combines different data types.

• Compare integer division with floating-point division.

• Use explicit casts where a fractional result is required.

• Demonstrate narrowing conversion and rounding behaviour.

• Use one consistent rule: convert rupees to paise by rounding to the nearest paise.

Program / Source Code

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    int units = 5;
    int count = 2;
    double  price  =  10.75;
    long big = 3000000000L;

    printf("Integer division: %d\n", units / count);
    printf("Floating division: %.2f\n", (double)units / count);
    printf("Mixed expression: %.2f\n", price + units);
    printf("Narrowed value: %d\n", (int)big);

    /* Round a rupee amount to the nearest paise */
    double charge = 12.345;
    long paise = (long)round(charge * 100.0);
    printf("Rounded paise: %ld\n", paise);
    return 0;
}
```

Output and Three Unexpected Results

```
Integer division: 2
Floating division: 2.50
Mixed expression: 15.75
Narrowed value: implementation-dependent result
Rounded paise: 1235
```

1. 5 / 2 gives 2: both operands are integers, so the fractional part is discarded. Use (double)5 / 2 to obtain 2.50.

2. Adding int and double: 10.75 + 5 gives 15.75 because the integer is converted to double before addition.

3. Casting a large long to int: 3000000000 may be outside the range of a 32-bit int. The converted result is implementation-defined when it cannot be represented; do not rely on it.

Rounding rule: use round(amount * 100.0) to round to the nearest whole paise. This example requires linking the math library on systems where round() is in libm: gcc task7.c -lm -o task7.

Alternative Concept and Complexity Analysis

Using floating point for all calculations and rounding only once at the end can allow small binary representation errors to accumulate. If each of n bill items is rounded separately, the maximum rounding difference per item is approximately half a paise; the total absolute difference can therefore grow up to about n × 0.5 paise in a worst-case bound. Integer paise gives exact addition for whole-paise amounts and uses constant time and constant space per value. Floating-point arithmetic is also O(1) per value, but introduces representation and rounding considerations. For money, integer paise with a documented rounding rule is generally easier to validate.

Result

The examples demonstrate integer division, implicit conversion, narrowing casts and rounding. Integer paise and a consistent nearest-paise rule provide predictable currency calculations.

TASK 8 — TRACE THE COMPILATION PROCESS

Task / ProblemStatement

FollowaCprogramthrough preprocessing, compilation to assembly, object-file generation and linking. Identify what the preprocessor changes and distinguish compile-time, link-time and run-time errors.

Aim

Tounderstand the stages of the GCC compilation process and use compiler commands to inspect each stage.

Concept / Theory

ACprogramnormallypasses through four major stages: preprocessing, compilation, assembly and linking. Preprocessing expands macros and includes header contents. Compilation translates the preprocessed C code into assembly. The assembler creates an object file. The linker combines object files and required libraries to produce an executable.

Procedure / Algorithm

• Save the source code astask8.c.

• Run gcc -E to view the preprocessed source.

• Run gcc -S to generate assembly code.

• Run gcc -c to generate an object file.

• Run gcc to link the object file and create an executable; then run it.

Sample Program

```
#include <stdio.h>
#define VALUE 25

int main(void)
{
    int number = VALUE;
    printf("Number = %d\n", number);
    return 0;
}
```

Compilation Commands and Evidence

```
gcc -E task8.c -o task8.i     # preprocessed source
gcc -S task8.i -o task8.s     #assembly source
gcc -c task8.s -o task8.o     #object file
gcc   task8.o   -o   task8    #linked executable
./task8
Number = 25
```

The commands can also be run directly as `gcc -S task8.c` and `gcc -c task8.c`; GCC performs the required earlier stages automatically.

What the Preprocessor Changes

The preprocessor expands #include <stdio.h> by inserting declarations from the standard header, replaces the macro VALUE with 25, and handles conditional-compilation directives. The preprocessed file contains expanded source and line-control markers.

Types of Errors and Warnings

Compile-time error: invalidCsyntaxoranundeclared variable, such as writing `int x = ;`. The compiler cannot produce valid object code.

Link-time error: calling a function whose definition is missing, such as declaring void display(void); but not providing its definition or linking the correct object file. The linker reports an undefined reference.

Run-time error: a problem while the executable runs, such as attempting to use an invalid pointer. The program may compile and link successfully but fail during execution.

Warnings: compile with gcc -Wall -Wextra -std=c11 task8.c -o task8. These options can identify issues such as unused variables, missing return statements in some contexts, or suspicious conversions. Fix warnings by correcting the source rather than simply suppressing them.

Alternative Concept and Complexity Analysis

If the program is split into input.c, tariff.c and report.c with a shared header, each source file is compiled separately into an object file and then linked. After changing only tariff.c, only tariff.c normally needs recompilation; the other object files can be reused, followed by relinking. In a single-file build, changing the file requires recompiling the whole translation unit. Separate compilation reduces rebuild time for larger projects, but object files require additional disk space. The total object-file space is approximately the sum of the object sizes.

Result

The preprocessing, compilation, assembly and linking stages were traced using GCC commands. Compile-time, link-time and run-time errors were distinguished, and the benefit of separate compilation was explained.

TEST CASES / VALIDATION TABLE

The following cases validate the key behaviour described in Tasks 4–8. The actual-output column records the expected result from the sample programs; confirm by running the code in your own GCC environment.

No.	Input / Scenario	Expected Output / Behaviour	Actual Output	Status
1	Task 4: rupees = 12	1200 paise	1200 paise	Pass
2	Task 4: SQUARE(4+1)	25 with fully parenthesised macro	25	Pass
3	Task 5: normal build	No DEBUG lines; total 210.00	Total = 210.00	Pass
4	Task 5: build with -DDEBUG	DEBUG amount and tax lines appear	DEBUG lines + total	Pass
5	Task 6: input 16250	Rs. 162.50	Rs. 162.50	Pass
6	Task 6: input abc	Invalid input message	Invalid input	Pass
7	Task 7: 5 / 2	Integer result 2	2	Pass
8	Task 7: (double)5 / 2	Floating result 2.50	2.50	Pass
9	Task 7: 12.345 rupees	Rounded to 1235 paise	1235	Pass
10	Task 8: sample program	Number = 25	Number = 25	Pass

Final Submission Checklist

- ■ Studentandcourse details completed.
- ■ Task statement, aim, theory and algorithm included for Tasks 4–8.
- ■Program code and sample output included.
- ■ Macro expansion and header-file debugging concepts explained.
- ■Input validation and format specifiers covered.
- ■ Type conversion examples and rounding rule explained.
- ■ GCC compilation stages and error categories explained.
- ■ Test cases included.

Submission note: The screenshots supplied show the assignment requirements only. This PDF contains prepared solutions and illustrative outputs, not captured screenshots of actual compiler execution. Add your own terminal screenshots if required by the instructor.

Assignment Overview

Grama Jyothi billshouseholds using previous and current meter readings. The program must calculate consumption, handle a 16-bit meter rollover, apply a fixed charge and progressive tariff slabs, validate inputs, and avoid monetary rounding errors. The system is also evaluated for growth from a village to a district and for smart meters that report every 15 minutes.

Tariff used in this report

Item	Rule
Fixed charge	■50.00 per bill
First 100 units	Free energy charge
Units 101–200	■2.25 per unit
Units above 200	■4.50 per unit
Meter range	0 to 65,535; rollover range = 65,536

Important implementation decision

Money is stored in paise as an integer. This avoids binary floating-point rounding errors. The meter readings are validated before subtraction. The code below assumes that a lower current reading indicates one rollover; in a real deployment, a meter-reset event should be checked separately so that a reset is not mistaken for rollover.

TASK 9 — Implement and Test the Billing Program

Aim: Implementacompletebillingprogramandvalidatenormalreadings,tariffboundaries, rollover, and invalid inputs.

Concept / Theory

The program reads previous and current meter values, checks their ranges, calculates consumed units, then applies the tariff slabs and fixed charge. A 16-bit meter has 65,536 possible values (0–65,535). When it rolls over, consumption is calculated as (65,536 – previous reading + current reading).

Algorithm

1. Read previous and current readings. 2. Reject non-numeric values or readings above 65,535. 3. If current ≥ previous, subtract normally; otherwise calculate rollover consumption. 4. Start the bill at 5,000 paise. 5. Charge 225 paise per unit for units 101–200 and 450 paise per unit above 200. 6. Display units and bill in rupees and paise.

C source code (complete program)

```
#include <stdio.h>
#include <limits.h>

#define METER_MAX 65535U
#define    METER_RANGE    65536U
#define FREE_UNITS 100U
#define SLAB2_LIMIT 200U
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define FIXED_CHARGE_PAISE 5000L

int main(void)
{
    unsigned int previous, current, units;
    long long bill_paise = FIXED_CHARGE_PAISE;
    unsigned int slab2_units = 0U;

    printf("Enter previous meter reading (0-65535): ");
    if (scanf("%u", &previous) != 1 || previous > METER_MAX) {
        printf("Invalid previous reading.\\n");
        return 1;
    }

    printf("Enter current meter reading (0-65535): ");
    if (scanf("%u", &current) != 1 || current > METER_MAX) {
        printf("Invalid current reading.\\n");
        return 1;
    }

    if (current >= previous)
        units = current - previous;
    else
        units = METER_RANGE - previous + current;

    if (units > SLAB2_LIMIT)
        bill_paise += (long long)(units - SLAB2_LIMIT) * RATE3_PAISE;

    if (units > FREE_UNITS) {
        slab2_units = (units < SLAB2_LIMIT)
        ? units - FREE_UNITS
        : SLAB2_LIMIT - FREE_UNITS;
        bill_paise += (long long)slab2_units * RATE2_PAISE;
    }

    printf("Units consumed: %u\\n", units);
    printf("Total electricity bill: Rs. %lld.%02lld\\n",
           bill_paise / 100, bill_paise % 100);
    return 0;
}
```

Sample output — normal reading

```
Enter previous meter reading (0-65535): 1200
Enter current meter reading (0-65535): 1530
Units consumed: 330
Total electricity bill:Rs. 950.00
```

Sample output — rollover

```
Enter previous meter reading (0-65535): 65400
Enter current meter reading (0-65535): 120
```

```
Total electricity bill: Rs. 527.00
```

Rollover bill: Rs. 50 fixed + 100 x Rs. 2.25 + 56 x Rs. 4.50 = Rs. 527.00. The sample output above is the expected result.

Validation table

Case	Previous	Current	Expected units	Expected bill / result
Zero use	500	500	0	■ 50.00
Exactly 100 units	1000	1100	100	■ 50.00
Exactly 200 units	1000	1200	200	■ 275.00
150 units	1000	1150	150	■ 162.50
350 units	1000	1350	350	■ 950.00
Rollover	65400	120	256	■ 527.00
Invalid range	70000	10	Rejected	Invalid previous reading
Text input	abc	120	Rejected	Invalid previous reading

Analysis / Discussion

The program uses integer paise to calculate the bill exactly. The first 100 units are free, the next 100 units cost ■2.25 each, and every unit above 200 costs ■4.50. Input checks prevent readings outside the meter range from being used. The rollover formula correctly calculates 256 units for readings 65,400 and 120.

Result

The billing program calculates consumption and charges using the stated tariff. Boundary values and rollover are included in the validation plan. Run each test on the compiler and add actual terminal screenshots before submission.

TASK 10— Compare Expectations with Execution

Aim: Compare theexpectedbehaviourofdatatypesandconstantswithactualexecution and assess total accumulation at different scales.

Data type choices

Data	Recommended type	Reason
Meter readings	unsigned int	Non-negative values from 0 to 65,535; validate the range explicitly.
Units consumed	unsigned int	Maximum one-rollover consumption is 65,535.
Tariff rates / bill in paise	long long	Integer arithmetic avoids floating-point currency rounding and provides a wide accumulation range.
Household count	size_t or unsigned long long	Suitable for counting records; validate the chosen range.
Tariff constants	#define or const	One source of truth makes revisions easier.

Expected versus observed behaviour

For a single bill, integer paise and floating-point rupees may appear to agree for simple values. Across thousands of bills, repeated floating-point calculations can accumulate small representation errors. Integer paise gives an exact total as long as the integer type does not overflow. An int or long may be only 16 or 32 bits on some systems, so the C standard's minimum guarantees must be checked using limits.h. long long has at least 64 bits in standard C implementations that support C99 or later.

Scale and overflow analysis

Scale	Households	What to check
Village	4,000	A 64-bit integer total is ample for ordinary bills.
Taluk	60,000	Use a wide integer total; do not assume int is enough for every possible tariff.
District	800,000	Use long long for total paise and check overflow before accumulation.

Alternative concept: use double for the total

A double has a large range and is useful for scientific calculations, but decimal currency values such as 2.25 are not always represented exactly in binary floating point. It can therefore introduce small precision differences when totals are accumulated. Integer paise is preferred for billing. Both approaches need approximately constant time per addition, O(1), and one variable of storage, O(1); the main difference is exactness and overflow behaviour.

Conclusion

Use integer paise for each bill and a sufficiently wide integer type for the total. Measure the actual results on the target compiler, print type limits with limits.h if required, and compare totals at 4,000, 60,000, and 800,000 records. Do not claim a measured difference until the program has been run.

TASK 11 — Recommend and Defend the Final Design

Aim: Recommendthedatatypes,constants,andtariffconfigurationmethodforareliablebilling system.

Recommended design

1. Use unsigned integer types for validated meter readings and consumed units. 2. Store money as integer paise. 3. Use long long for the bill and aggregate totals, with an overflow check before adding each bill. 4. Define the meter range and tariff rules in one place. 5. Separate input validation, consumption calculation, tariff calculation, and output into functions as the program grows.

Tariff revision

For a small fixed tariff, symbolic constants are simple and fast. However, changing a #define value requires recompilation and redistribution. Since the co-operative expects tariff revisions, a configuration file is more suitable. At program start, the program should read the fixed charge, slab limits, rates, and effective date, validate that values are non-negative and ordered, and reject an invalid configuration. Keep a versioned backup and an audit record for every approved revision.

Limitations

The simple design assumes a single rollover whenever the current reading is less than the previous reading. It cannot distinguish a genuine rollover from a meter reset, data-entry mistake, or multiple missed cycles. It also uses one tariff for the whole billing period and does not yet account for taxes, subsidies, meter faults, or time-of-day usage. These cases require additional rules and records.

Will it remain correct at district scale?

The bill calculation for one household remains O(1) because it uses a fixed number of arithmetic operations. Processing n household bills takes O(n) time. A basic program can handle 800,000 bills if memory, input/output, and integer ranges are managed correctly. A production system should stream records or process batches, validate each record, log rejected records, and use checked wide-integer aggregation.

Alternative concept and complexity

If slab limits and rates are stored in arrays and processed in a loop, the tariff function takes O(s) time for s slabs and O(s) extra space for the tariff table. With only three slabs, fixed if-statements are simple and take O(1) time and O(1) space. The array approach is easier to revise and extend to many slabs. Reading t tariff values from a configuration file takes O(t) start-up time and O(t) storage, but avoids recompilation after each approved tariff change.

Final recommendation

Retain integer paise and wide integer totals, but modify the program to separate the tariff configuration from the source code. Use a validated configuration file for rates, explicit error handling, a rollover/reset check, and automated boundary tests. The recommendation should be supported by actual compiler runs and measured totals recorded in the submission.

TASK 12 — Re-evaluate and Adapt the Design for Smart Meters

Changedrequirement: Smart meters report watt-hoursevery15 minutes(35,040readingsperhouseholdper year). Units consumed between 6 PM and 10 PM cost 20% more. Tariff values must be changeable without recompiling.

Decision: MODIFY the original design

The original billing logic can be retained as a starting point, but the input model and tariff calculation must be modified. Instead of relying only on two monthly meter readings, the system should process time-stamped watt-hour interval records. The tariff configuration should be loaded from a file or database at start-up, with validation and a version/effective date.

Proposed processing steps

1. Read each interval record: meter ID, timestamp, and watt-hours. 2. Validate the meter ID, timestamp, and non-negative energy value. 3. Convert watt-hours to billing units (kWh) using 1 kWh = 1,000 Wh, retaining integer Wh internally where possible. 4. Identify whether the interval falls between 18:00 and 22:00 local time. 5. Apply the 20% peak multiplier to the energy rate for those intervals. 6. Aggregate energy charges in a sufficiently wide integer representation, with a documented rounding rule. 7. Add the fixed charge once per billing period, not once per interval. 8. Produce a bill summary and an audit trail.

Example of peak rate

The standard rate above 200 units is ■4.50 per unit. A 20% increase gives ■4.50 × 1.20 = ■5.40 per unit during the peak period. The standard ■2.25 rate becomes ■2.70 during the peak period. The first-100-units-free rule must be clarified by the co-operative: this report assumes the free allowance applies to total monthly units before the time-of-day multiplier is applied to chargeable units.

Data volume and complexity

35,040 readings perhouseholdper year means approximately 140,160,000 readings for 4,000 households, 2,102,400,000 readings for 60,000 households, and 28,032,000,000 readings for 800,000 households. The system should process records in a streaming or batched manner instead of storing all readings in memory. Processing N readings takes O(N) time; a streaming approach can use O(1) working memory per record, apart from buffers and the tariff table.

Configuration-file versus compiled constants

Approach	Advantages	Limitations
Compile-time constants	Fast, simple, no file parsing at start-up	Every tariff revision requires recompiling and redistributing the program
Configuration file	Tariffs can change without recompilation; supports versioning and effective dates	Must validate file contents; adds start-up I/O and configuration-error handling

Risks and controls

Use secure access to tariff files, validate all rates and limits, log configuration changes, synchronise meter timestamps, detect missing or duplicate intervals, and keep the original readings for audit. The peak window must use the co-operative's official time zone. Missing data should be flagged rather than silently treated as zero. A reset or rollover should be explicitly identified where possible.

Final result

The original program should be modified, not used unchanged. Its integer-money principle and validation approach remain useful, while time-stamped interval processing, peak-rate calculation, configurable tariffs, and large-scale batch processing are required for smart meters.

Compilation, Execution and GitHub Submission

Compiler

Use GCC with C99 or later. Save the source code as **billing.c**. Compile using:

```
gcc -std=c11-Wall-Wextra-Wpedantic billing.c -o billing
```

Run on Windows

```
billing.exe
```

Run on Linux / macOS

```
./billing
```

Suggested folder structure

```
CSA02-1971260029-Assignment1/
  README.md
  Task01/
    billing.c
    test-cases.txt
    output-screenshots/
  Task02/
    type-analysis.md
    totals-test.c
  Task03/
    design-recommendation.md
  Task04/
    smart-meter-design.md
    sample-config.txt
```

README.md checklist

Include student name, register number, compiler name/version, exact compile command, run steps, test cases, results, and the repository URL. Commit progressively with at least one meaningful commit per task. Add the course faculty as a collaborator and submit the working GitHub link in the assignment form and on the report cover.

Sample test summary

Test	Input / case	Expected result
1	Previous 500, current 500	0 units; ■ 50.00
2	100 units consumed	■50.00
3	150 units consumed	■162.50
4	200 units consumed	■275.00
5	350 units consumed	■950.00
6	Previous 65400, current 120	256 units; ■527.00
7	Reading greater than 65535	Reject input
8	Non-numeric input	Reject input

Final submission checklist

■ Source code compiles without warnings. ■ Run every test and record actual results. ■ Insert genuine output screenshots from your own compiler run. ■ Complete the README. ■ Create task-wise folders. ■ Commit each task separately. ■ Verify the repository link and collaborator access. ■ Upload the PDF to the assignment portal.

Important: This completed report is based on the uploaded assignment template and visible task instructions. The sample outputs are illustrative; execute the program yourself and add your real output screenshots and measured totals before final submission.